

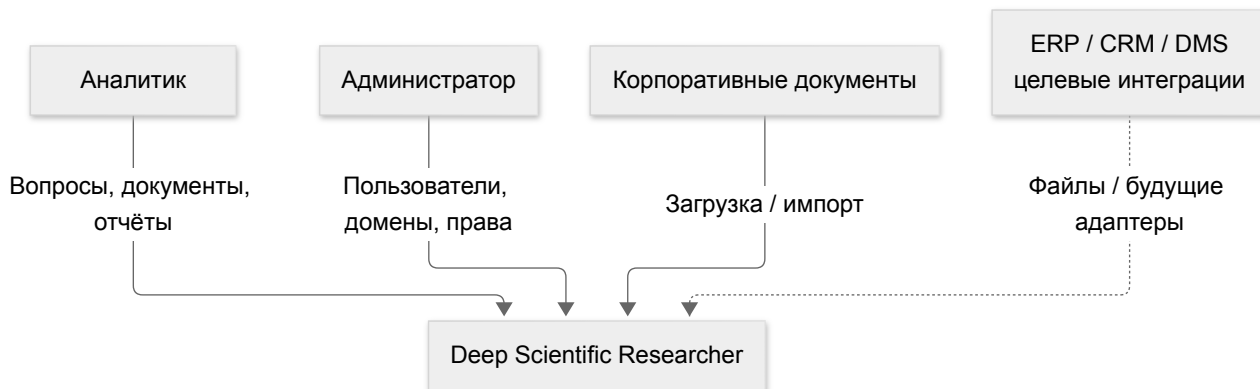
Содержание

1	Архитектура MVP	2
1.1	C1. Контекст	2
1.2	C2. Контейнеры и разделение плоскостей	3
1.3	C3. Компоненты агента	4
1.4	C4. Код ядра агента	5
1.5	B1. Stateful workflow	6
1.6	R1. GraphRAG и контроль контекста	7
1.7	F1. Data Flow и граница доверия	8
1.8	S1. Обработка запроса	9
1.9	D1. Физическое размещение	10
2	Модель данных	11
2.1	E1. Документы, права и GraphRAG	11
2.2	E2. Сессии, очередь и результаты	12
3	ADR: выбор стека	13
3.1	ADR-001. Закрытый контур	13
3.2	ADR-002. Stateful agent	13
3.3	ADR-003. GraphRAG	13
3.4	ADR-004. Self-hosted данные	13
3.5	ADR-005. Serving и квантование	13
3.6	ADR-006. Сложные PDF	13
3.7	ADR-007. Security-by-Design	13
3.8	ADR-008. Наблюдаемость и секреты	13
3.9	ADR-009. Импорт корпуса	14
3.10	ADR-010. Проверяемый отчёт	14
4	Запуск и проверка	15
4.1	1. Подготовка вне закрытого контура	15
4.2	2. Конфигурация на сервере	15
4.3	3. Запуск	15
4.4	4. Приёмка	15
5	Нагрузочный отчёт — 04.10.2026	17
5.1	Полный pipeline	17
5.2	Inference отдельно от исследований	17
5.3	Качество, ресурсы и offline	17

1 Архитектура MVP

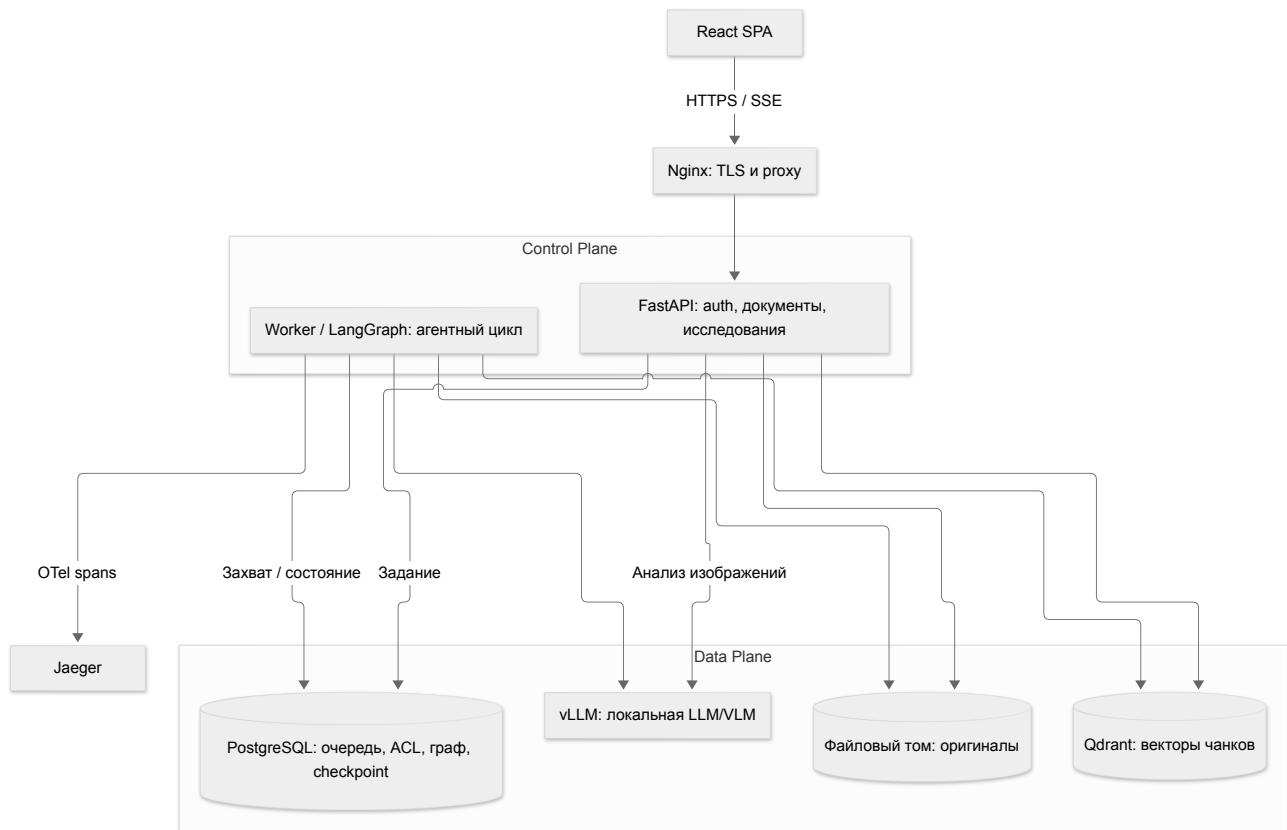
Система преобразует технические PDF, сканы и таблицы в отчёт на русском с проверяемыми цитатами. Профиль сдачи — локальные модели, APP_MODE=airgap, GRAPH_RETRIEVAL_ENABLED=true. Измерения и границы проверенного поведения — в [нагрузочном отчёте](#); выбор технологий — в [ADR](#).

1.1 C1. Контекст



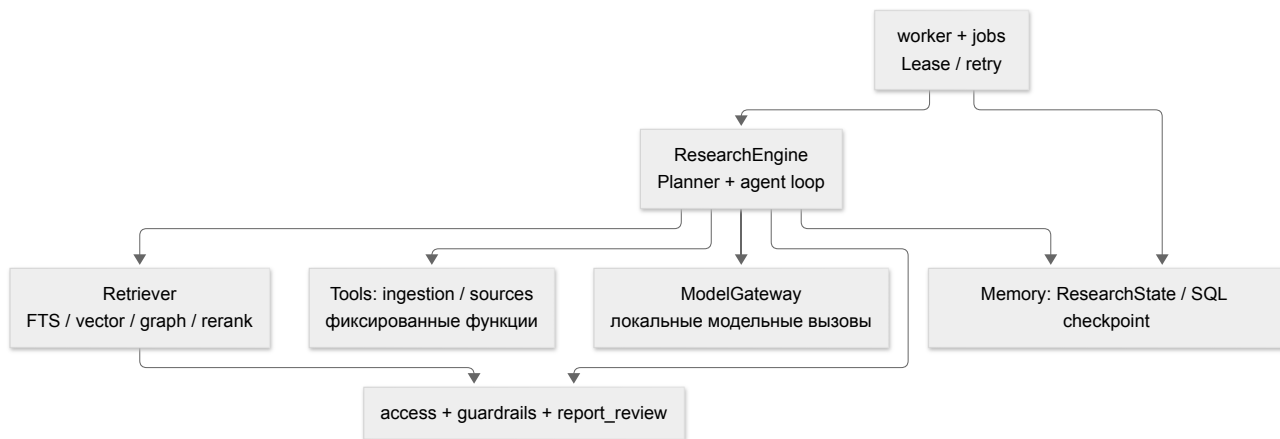
C1. Граница системы. В MVP документы загружаются через API/интерфейс; прямые ERP/CRM-коннекторы не реализованы. Интернет-сбор исходного connected-профиля не используется при сдаче.

1.2 C2. Контейнеры и разделение плоскостей



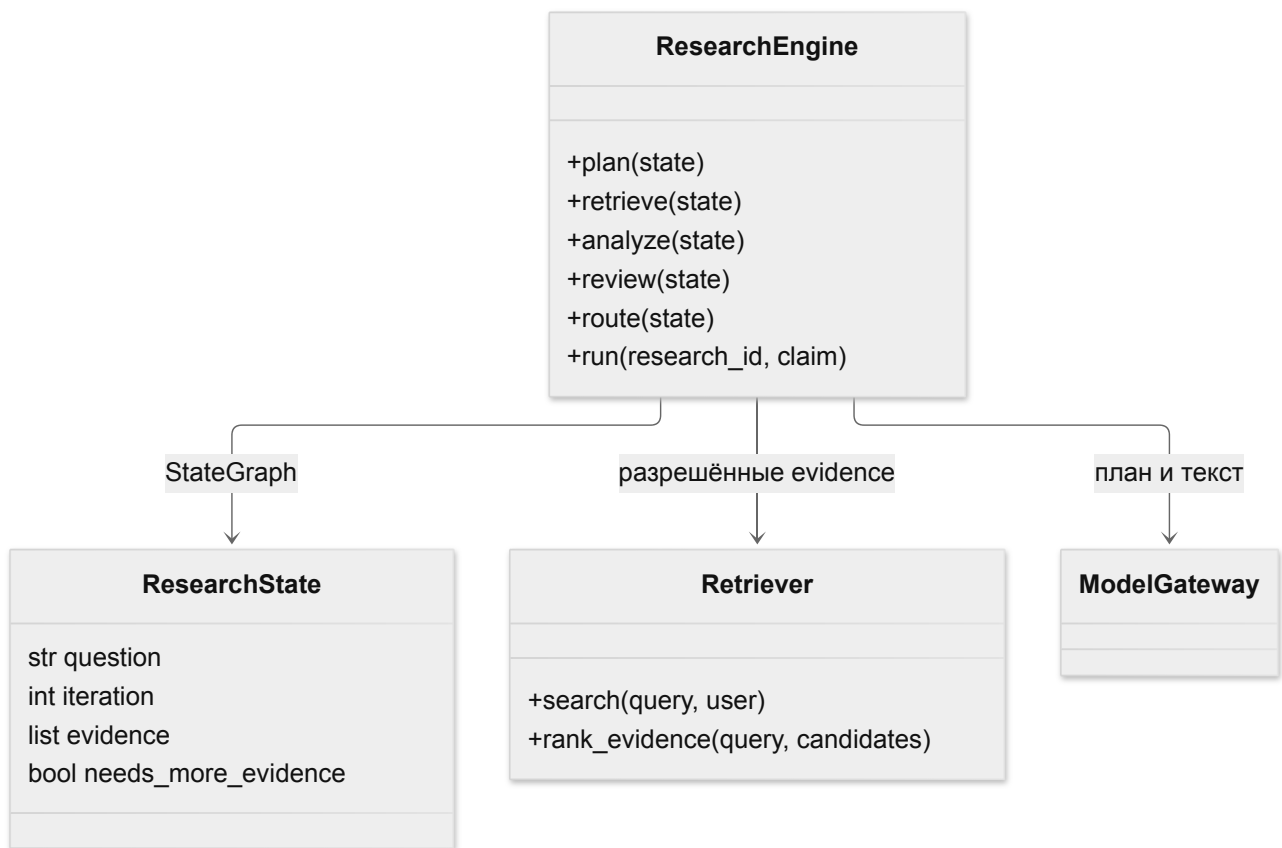
C2. Процессы и хранилища. API и worker общаются через SQL-очередь. Embeddings и rerank работают локально внутри приложения. В измеренном стенде использован Jaeger. Prometheus, Grafana и Langfuse доступны дополнительными профилями [infra/](#).

1.3 С3. Компоненты агента



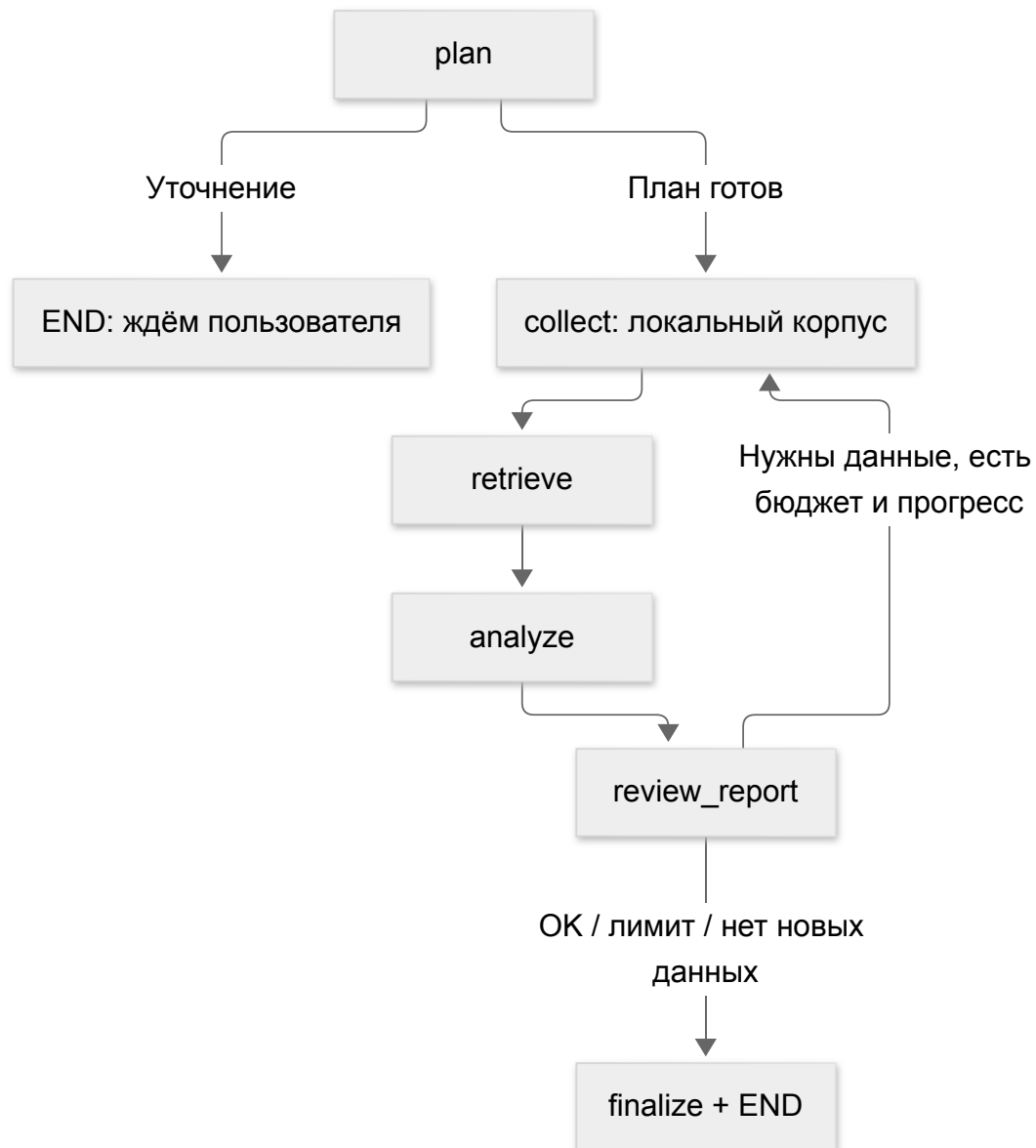
С3. Ответственность компонентов. Модель предлагает план и текст; код выполняет поиск, проверяет права и сохраняет результат. Memory — состояние run и PostgreSQL checkpoint, а не отдельный memory-сервис. В airgap внешний сбор пропускается. [Реализация](#).

1.4 C4. Код ядра агента



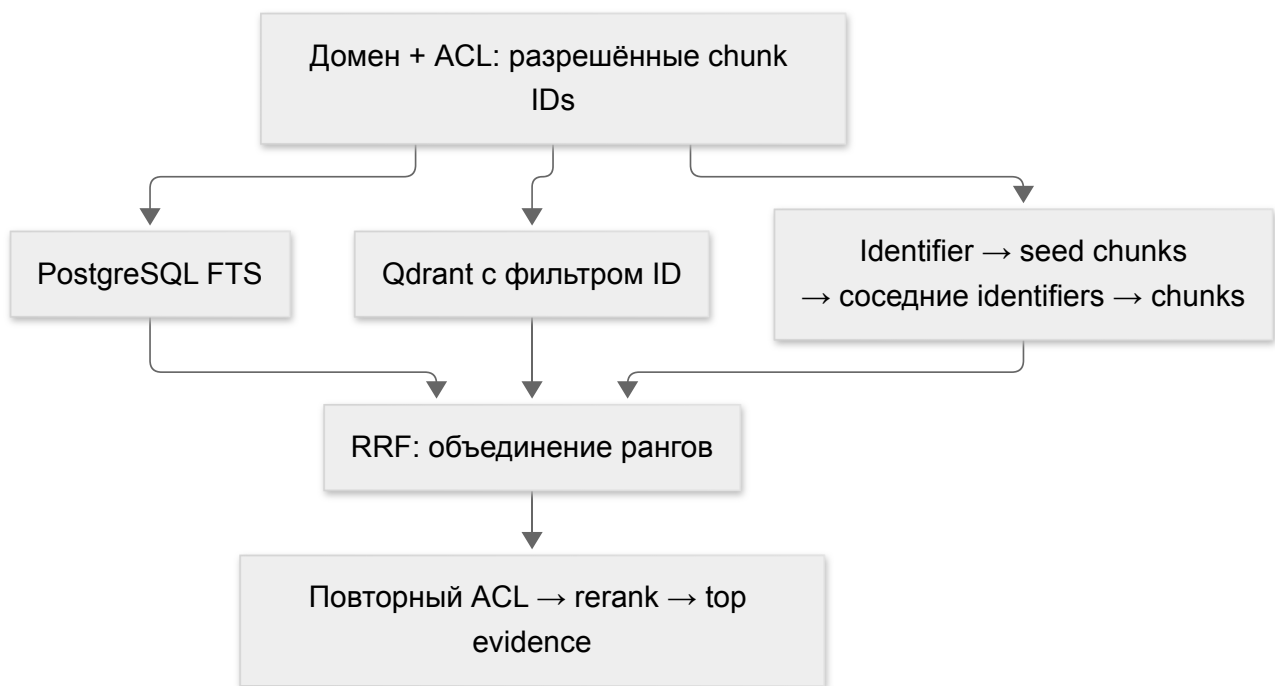
C4. Существующие классы и методы. Поля/методы показаны выборочно; полный контракт — в [research.py](#), [retrieval.py](#) и [vision.py](#).

1.5 B1. Stateful workflow



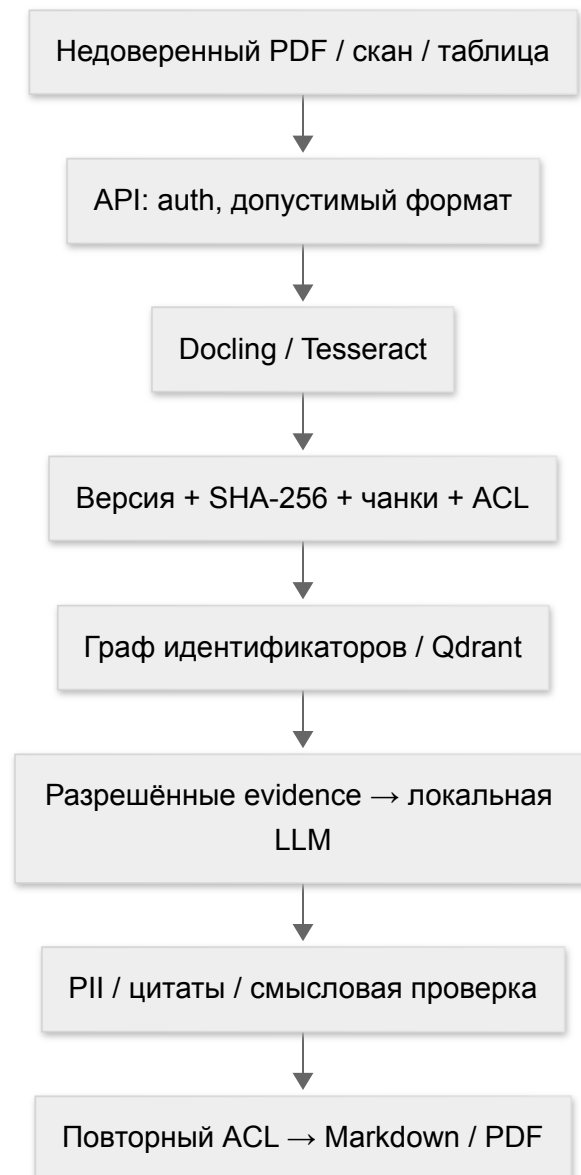
B1. Цикл LangGraph. `max_research_steps` ограничивает поиск; `checkpoint` привязан к `run_id`. Worker использует `lease` и `heartbeat`; `retry` продолжает тот же `run`. Отчёт и `completed` фиксируются атомарно. SSE передаёт статусы/этапы, а не токены LLM.

1.6 R1. GraphRAG и контроль контекста



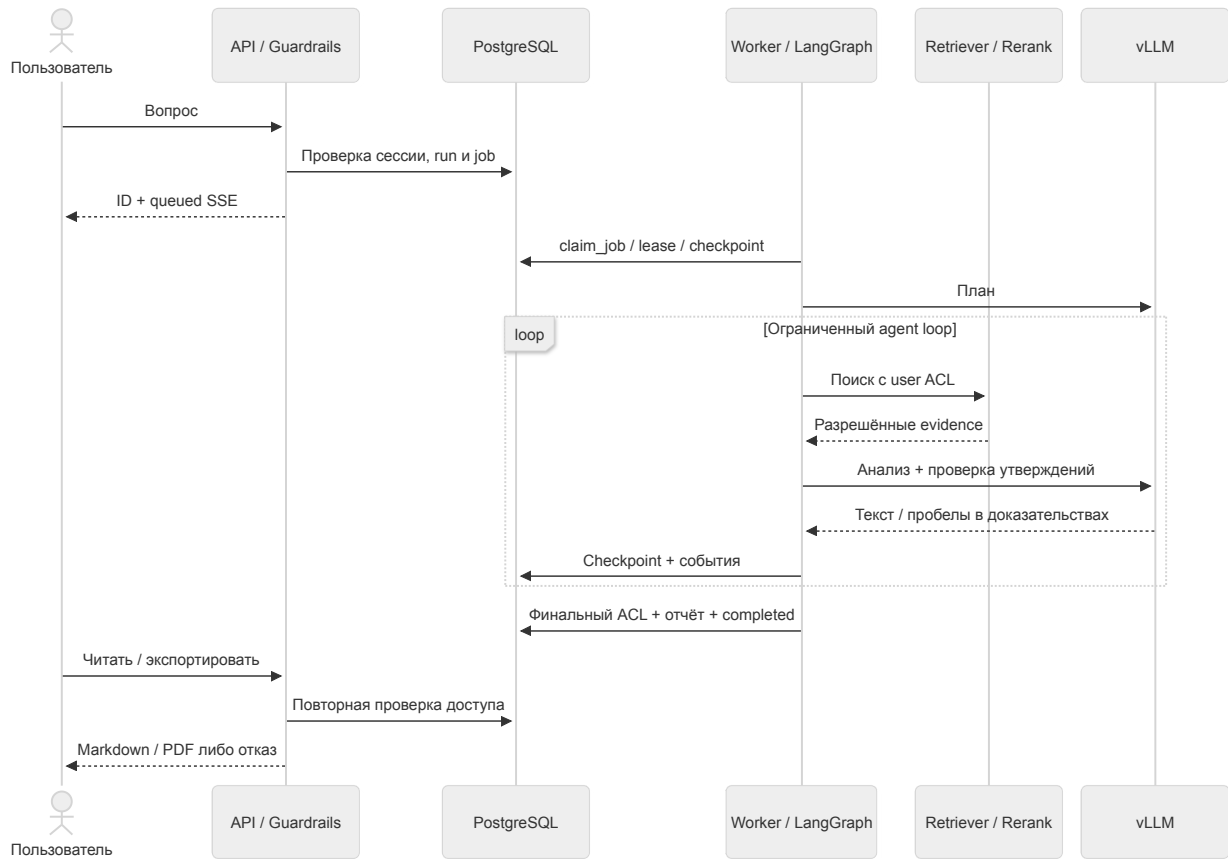
R1. Поиск не расширяет права. Онтология: Document → Chunk → Identifier; связи хранятся в PostgreSQL. Закрытый чанк не может быть мостом графа. Qwen3 Reranker использует yes/no logits и порог 0.1; произвольная семантическая онтология не строится. [Граф](#), [reranker](#), [ER](#).

1.7 F1. Data Flow и граница доверия



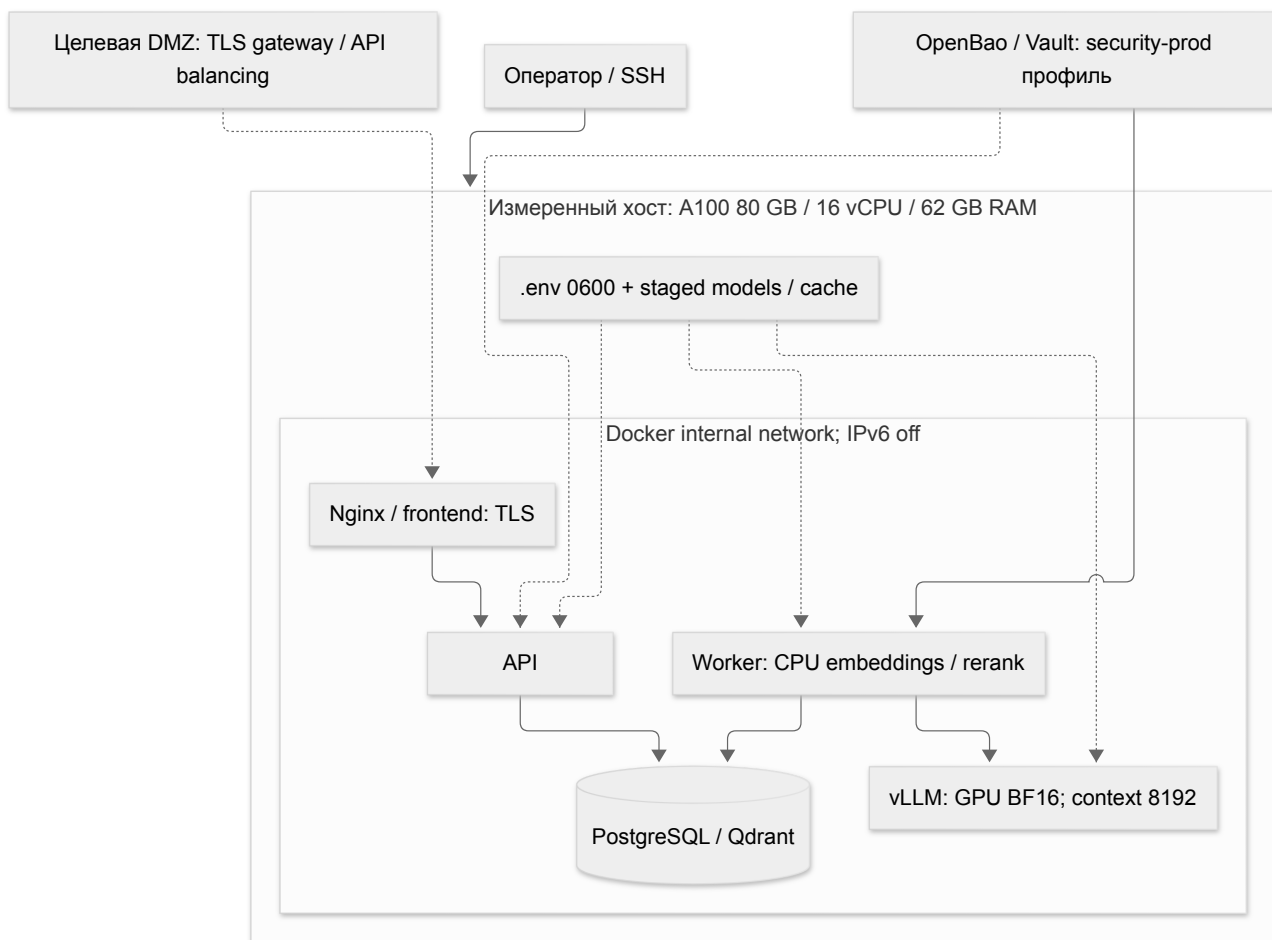
F1. Документ — данные, а не инструкция агенту. Оригинал и происхождение цитат сохраняются. Проверка удаляет неподтверждённые утверждения; недостаток данных приводит к уточнению/отказу. PII-фильтр покрывает email/телефоны, не полноценную DLP. Геометрию чертежей нужно сверять вручную.

1.8 S1. Обработка запроса



S1. Авторизация остаётся в коде. Отзыв доступа закрывает ранее выданные ссылки, источники и экспорты. SSE читается из PostgreSQL; выполнение инструментов не передаётся модели.

1.9 D1. Физическое размещение

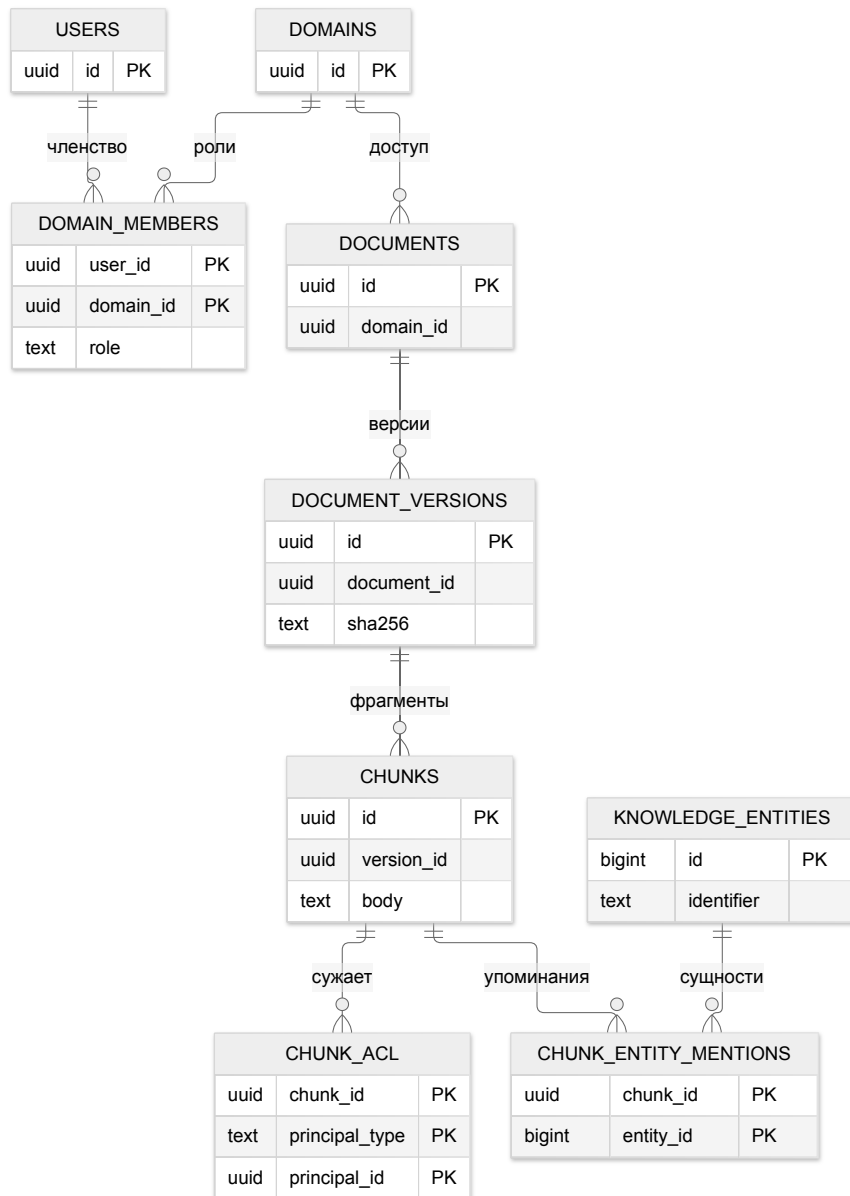


D1. Измеренное размещение и целевые расширения. Семь сервисов работают на одном хосте; `unless-stopped` обеспечивает повторный запуск после `reboot`. `API/worker/model` не имеют внешнего выхода; сам хост подключён. DMZ, балансировка нескольких API и OpenBao с TLS — целевой профиль, не заявленный результат `benchmark`. В нём приложение и GPU могут быть разнесены; правила `firewall` разрешают только внутренние зависимости. [Compose стенда](#), [секреты/профили](#).

2 Модель данных

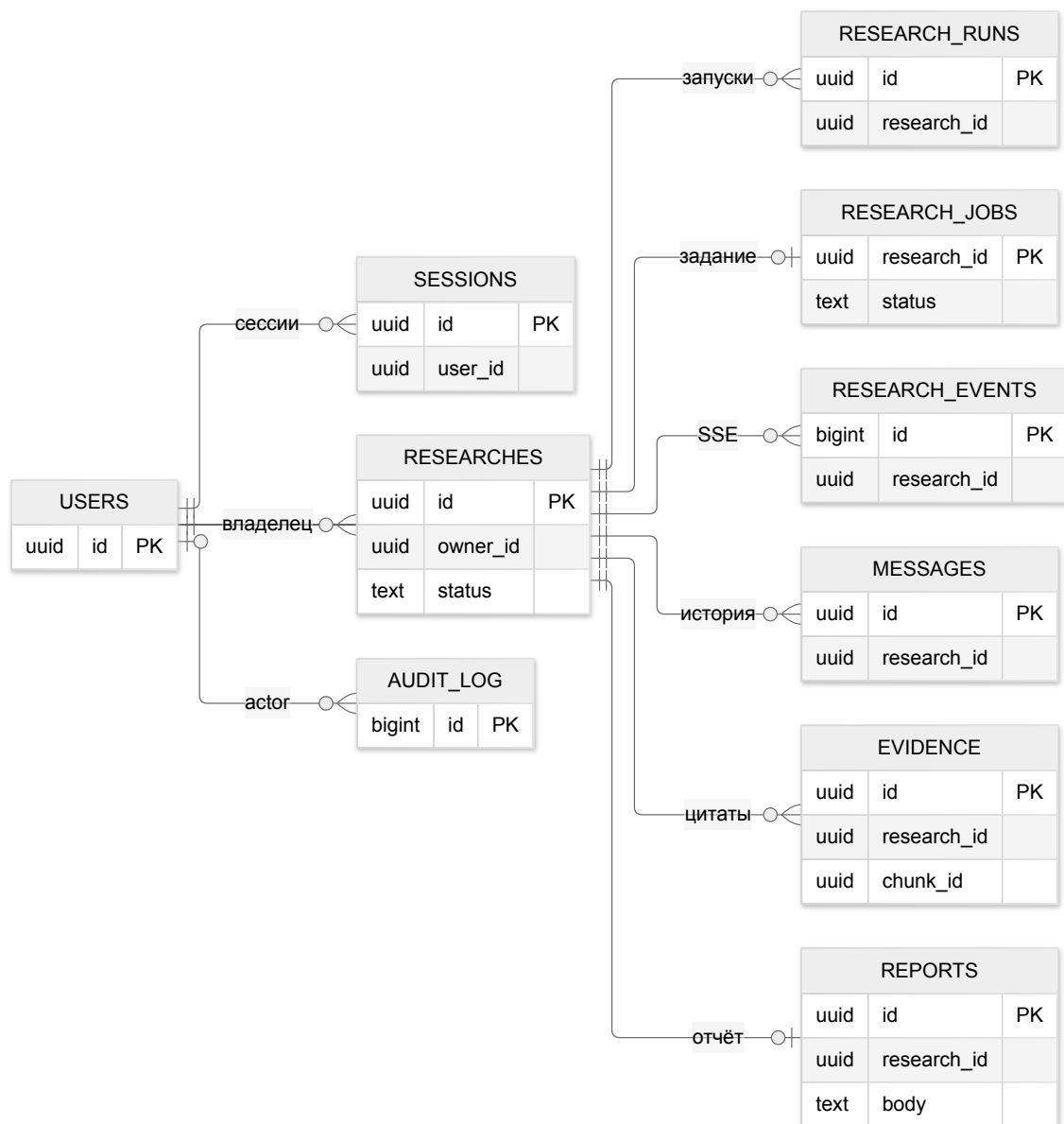
Два ER-вида раскрывают Data Plane из C2. Показаны ключевые поля и связи; полный DDL, FK, индексы и миграции — [backend/schema.sql](#). Связи на обзорной схеме логические; она не заменяет DDL.

2.1 E1. Документы, права и GraphRAG



E1. Происхождение и доступ. Домен даёт базовое право, ACL чанка может только сузить его. principal_id полиморфный user/group; таблицы групп здесь опущены. Вектор Qdrant имеет UUID чанка, оригинал находится в файловом томе. Граф восстанавливается из разрешённых упоминаний; сущность сама по себе не открывает доступ к документу.

2.2 E2. Сессии, очередь и результаты



E2. Состояние и проверяемый ответ. evidence.chunk_id ссылается на CHUNKS из E1; доступ проверяется заново при чтении отчёта. Сессии содержат hashes токенов, аудит — метаданные без текстов документов/секретов. Таблицы checkpoint создаёт PostgreSQL checkpointер LangGraph. PDF генерируется из Markdown и отдельно не хранится.

3 ADR: выбор стека

Решения относятся к профилю сдачи. [Архитектура](#) показывает границы, [нагрузочный отчёт](#) — что реально проверено.

3.1 ADR-001. Закрытый контур

Решение: APP_MODE=airgap, локальные модели, предварительно загруженный корпус, offline-флаги и запрет egress. Внешние API не участвуют в сдаче. **Trade-off:** меньше актуальность корпуса, зато автономность и контроль данных; импорт выполняет оператор. Исходный connected-режим оставлен для разработки.

3.2 ADR-002. Stateful agent

Решение: LangGraph и PostgreSQL checkpoint; plan → retrieve → analyze → review → повтор/завершение, с бюджетом и lease. **Trade-off:** явное состояние сложнее линейного скрипта, но поддерживает повторный поиск, уточнение и восстановление. Свободная multi-agent оркестрация не нужна MVP. [Код](#).

3.3 ADR-003. GraphRAG

Решение: FTS + Qdrant + граф Document/Chunk/Identifier, RRF и rerank. Граф хранится в PostgreSQL и извлекается детерминированно из технических идентификаторов. **Trade-off:** проще восстановление и ACL, чем отдельный Neo4j с LLM-онтологией; ниже семантическая полнота. Польза качества требует размеченного A/B. [Код](#).

3.4 ADR-004. Self-hosted данные

Решение: PostgreSQL для прав/очереди/графа, Qdrant для векторов, том для оригиналов. **Trade-off:** Qdrant проще для одного индекса MVP, чем кластер Milvus; один индекс с обязательным фильтром UUID проще коллекций на пользователя. Индекс можно пересоздать из версий документов.

3.5 ADR-005. Serving и квантование

Решение: vLLM 0.30.0; на A100 измерен Qwen3.8-27B BF16, context 8192, prefix caching, GPU memory utilization 0.90. Embedding/reranker — локальные Qwen3 0.6B; reranker использует обученные yes/no logits. **Trade-off:** BF16 помещается на 80 GB; для меньшей GPU предусмотрен [AWQ-профиль](#), но он не измерен. GGUF потребовал бы отдельного serving-профиля; SGLang/TGI не сравнивались нагрузкой. Версии весов/image digest зафиксированы в raw evidence.

3.6 ADR-006. Сложные PDF

Решение: Docling для структуры/таблиц, Tesseract rus+eng для OCR, локальный VLM для отдельного анализа изображений. **Trade-off:** rupdф не распознаёт сканы; VLM на каждой странице дороже и может ошибаться в геометрии. Веса Docling доставляются заранее, инженерные размеры проверяет человек.

3.7 ADR-007. Security-by-Design

Решение: доменные роли + ACL чанков, фильтрация до retrieval/graph traversal и повторный контроль перед текстом, цитатами и экспортом. Источник не получает полномочий через prompt. **Trade-off:** SQL-список разрешённых ID прост для MVP, но требует пересмотра на большом корпусе. PII-фильтр email/телефонов ограничен; это не DLP. [Код](#).

3.8 ADR-008. Наблюдаемость и секреты

Решение: OTel → Jaeger; Prometheus/Grafana и self-hosted Langfuse — дополнительные infra-профили. Атрибуты без промптов/ответов/секретов. OpenBao с TLS предусмотрен security-prod; измеренный стенд использует .env 0600. **Trade-off:** Jaeger проще для demo, Langfuse добавляет зависимости; ни наличие конфигурации, ни trace не доказывают промышленную готовность.

3.9 ADR-009. Импорт корпуса

Решение: оригиналы, версии и SHA-256; публичные пакеты переносятся оператором, суммы сверяются перед индексированием. **Trade-off:** ручной импорт менее удобен web search, но соответствует закрытому контуру. SHA-256 проверяет целостность, а не подлинность отправителя.

3.10 ADR-010. Проверяемый отчёт

Решение: Markdown — источник, PDF — представление после повторного ACL. Механическая проверка цитат/чисел и LLM-проверка утверждений; неподтверждённый текст удаляется. **Trade-off:** можно потерять полезную формулировку/OCR-формулу, зато неподтверждённый ответ не считается успешным. PDF через ReportLab/MathText не исполняет модельный TeX. [Код](#).

4 Запуск и проверка

Профиль для сдачи и воспроизведения измеренного стенда — [infra/compose.benchmark.yaml](#): API, worker, frontend, PostgreSQL, Qdrant, vLLM, Jaeger. Стенд Linux + Docker Compose + NVIDIA Container Toolkit; A100 80 GB, 16 vCPU, около 64 GB RAM. Все сервисы имеют `unless-stopped`.

4.1 1. Подготовка вне закрытого контура

Заранее установить зависимости, собрать образы и доставить модели. Приватные `.env`, сертификаты, веса и документы в Git не входят.

```
poetry install --only main
poetry run python -m scripts.stage_benchmark_models \
  --root "$HOME/research-models"
docker build -t research-backend:submission -f backend/Dockerfile .
docker build -t research-frontend:submission frontend
```

Доставить pinned vLLM image и образы БД/Jaeger из Compose через `docker save/docker load`. Веса generation, embedding, reranker, docling должны лежать в `~/research-models`; сверить [manifest](#). Скрипт staging загружает embedding и reranker; generation и Docling доставить отдельно до изоляции.

4.2 2. Конфигурация на сервере

```
python3 -m scripts.init_benchmark_host
```

Скрипт создаёт `.env.benchmark` с правами 0600 и localhost TLS-сертификат; существующие секреты не заменяет. В этом файле выставить:

```
BENCHMARK_BACKEND_IMAGE=research-backend:submission
BENCHMARK_NETWORK_INTERNAL=true
GRAPH_RETRIEVAL_ENABLED=true
OPENBAO_ENABLED=false
```

`OPENBAO_ENABLED=false` относится только к изолированному учебному benchmark. Для целевого секретного хранилища использовать [OpenBao](#), TLS, read-only token file и профиль `security-prod` в [основном Compose](#). Dev root token не подходит рабочему контуру. Основной Compose также содержит Prometheus/Grafana; [Langfuse](#) подключается отдельно.

4.3 3. Запуск

```
docker compose --env-file .env.benchmark \
  -f infra/compose.benchmark.yaml up -d
```

Offline-флаги, пути локальных моделей, API key и TLS заданы Compose. Первый запуск модели занимает несколько минут. `/health/ready` проверяет PostgreSQL, не весь pipeline. Перед пользовательским запросом дождаться успешной генерации модели.

В internal-сети опубликованные loopback-порты Docker на измеренном хосте не обеспечили доступ из host namespace. Benchmark-клиенты запускаются с `--network container:research-benchmark-frontend-1`, URL `https://localhost:443`. Для браузерного demo нужен отдельный разрешённый ingress/туннель; работающий host port не заявляется. Не открывать model/DB в интернет.

4.4 4. Приёмка

Проверка	Воспроизводимый инструмент
Корпус: TXT, PDF, скан, таблица	prepare_benchmark_corpus.py
Фиксированный corpus score	isolate_benchmark_corpus.py
Качество и цитаты / ACL / injection	benchmark_quality.py , benchmark_security.py
GraphRAG и запрещённый мост	check_graph_retrieval.py
Startup и egress	check_offline_startup.py , check_runtime_isolation.py
RPS/latency и inference	benchmark_research.py , benchmark_inference.py

Параметры каждого инструмента — `python -m scripts.<имя> --help`. Startup-check требует остановленный model и запускает сервисы; его не применять во время исследований. Для проверки качества использовать synthetic accounts/корпус, не секретные рабочие документы. Raw результаты и статусы — в [нагрузочном отчёте](#).

Локальные тесты: `poetry run pytest -q`. Интеграции запускать только после создания отдельной тестовой БД и экспорта TEST_DATABASE_URL:

```
INTEGRATION_TESTS=1 DATABASE_URL="$TEST_DATABASE_URL" \  
poetry run pytest -q
```

Не указывать production-базу: тесты создают/удаляют свои fixtures. Последний полный прогон на изолированной БД — 86 passed. Для восстановления нужны согласованные backup PostgreSQL и оригиналов; Qdrant/граф можно переиндексировать. Целевые RPO/RTO не измерены.

5 Нагрузочный отчёт — 04.10.2026

Стенд: A100 80 GB PCIe, 16 vCPU Xeon Ice Lake, 62 GB RAM. **Профиль:** один worker, GraphRAG, Qwen3.8-27B BF16, vLLM 0.30.0, context 8192, prefix caching. Вход — synthetic документы о ДД-42; нагрузочный пользователь А имел доступ и к security-fixtures.

5.1 Полный pipeline

Concurrency	N / завершения	Успешный RPS	Отчётов/мин	p50 / p95, c
1	2 / 2	0.009854	0.59	100.43 / 102.52
2	4 / 4	0.009996	0.60	198.96 / 201.17

Практический результат: около **0.010 RPS**, или **0.60 отчёта в минуту**. Ошибок/таймаутов в шести запросах не наблюдалось. Один worker обслуживает очередь последовательно: на concurrency 2 queue p95 99.93 c, execution p95 99.83 c. Первый SSE p95 80 мс — событие queued, не первый токен.

N=2/4 и отсутствие длительного soak не устанавливают capacity/SLA; p95 при таком N близок максимуму. Наличие отчёта не означает автоматически прохождение проверки качества. [Raw baseline](#), [raw c2](#).

5.2 Inference отдельно от исследований

2104–2105 входных токенов, 128 выходных, N=16 на ступень, уникальный cache salt.

Concurrency	Выходных токенов/с	TTFT p95, c	Latency p95, c
1	24.62	0.605	5.20
4	73.09	2.350	8.15
16	139.95	9.357	14.63

[Raw inference](#). Эти токены/с не являются RPS полного GraphRAG pipeline.

5.3 Качество, ресурсы и offline

- **Quality 11/11** на отдельном корпусе: 7 документов / 9 чанков; основной ответ за 140.386 c. **Security 11/11, GraphRAG ACL 4/4** — узкие synthetic-проверки.
- За load-интервал: GPU SM peak 100%, host RAM 21.27 GiB, worker memory usage 11.72 GiB. Это sampled peaks, не непрерывный максимум.
- API/worker/model: все 9 egress probes заблокированы. После reboot model startup и генерация прошли; readiness около 202 c, с подготовленными весами и compiler cache. Сам host имеет интернет.
- Consumer AWQ, пустой cache, длительная нагрузка и выигрыш GraphRAG на размеченном корпусе не измерены. Исторический baseline со случайной головой reranker не используется для текущего результата.

[Quality](#), [security](#), [graph](#), [resources](#), [isolation](#), [startup](#), [runtime/source hashes](#). В [evidence index](#) перечислены остальные подтверждения и границы воспроизводимости.